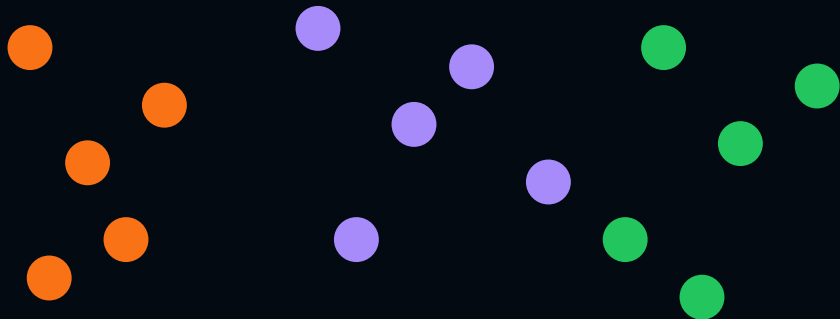


K-Means

Clustering Algorithm

Unsupervised Learning — Find Groups Without Labels

- R.M.K. Engineering College
First Year | Batch 2025–2029



Slides cover

- What is Clustering?
- What is K-Means?
- Step-by-step working
- Visual walkthrough
- Distance formula
- Worked example
- Iteration 1,2,3
- Convergence
- Choosing K (Elbow)
- Evaluation metrics
- Pros, Cons & Uses
- Summary

Clustering — Grouping Similar Things Together

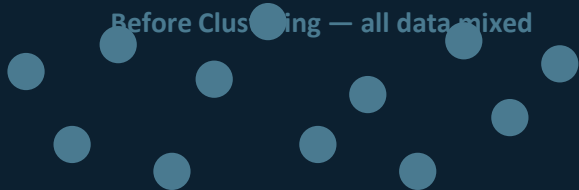
Clustering is the task of grouping data points so that points WITHIN the same group are more similar to each other than to points in other groups — WITHOUT any labels or correct answers.



Real-life analogy you already know:

A librarian organises 1000 books into sections — Fiction, Science, History, Sports — without being told which book belongs where. She groups books by similarity. That is EXACTLY what K-Means Clustering does with data!

Before Clustering — all data mixed



No groups — all same colour

K-Means
(K=3) →

After Clustering — 3 distinct groups



3 colour-coded clusters + centroids (X)

K-Means — The Algorithm Explained Simply

K-Means divides a dataset into K clusters. Each data point is assigned to the cluster whose CENTROID (centre point) is nearest to it. Centroids are updated repeatedly until the assignments stop changing.

What does 'K' mean?

K = the NUMBER OF CLUSTERS you want. You must decide this BEFORE running the algorithm. It is a hyperparameter — like choosing how many drawers to sort your clothes into before you start sorting.

What is a 'Centroid'?

Centroid = the AVERAGE position of all points in a cluster. It is the mathematical centre of the cluster — like the centre of gravity. It may not be an actual data point.

K =
2

Classify emails: Spam vs Not Spam

K =
3

Customer types: Premium, Regular,
Occasional

K =
4

News: Sports, Tech, Politics, Business

K = number of clusters (YOU decide) | Centroid = average centre of each cluster

K-Means Algorithm — Step by Step

1

Choose K

Decide how many clusters you want (e.g. $K=3$). This is set by you before training begins.

2

Initialise K centroids

Randomly place K centroid points anywhere in the feature space as starting positions.

3

Assign each point

Calculate the Euclidean distance from every data point to EACH centroid. Assign the point to its NEAREST centroid.

4

Recalculate centroids

For each cluster, calculate the NEW centroid = the MEAN (average) of all data points currently assigned to it.

5

Check for convergence

Did any points change cluster? If YES → repeat steps 3 and 4. If NO → algorithm has converged. STOP.

6

Final clusters

Output: K clusters with their centroids. Every data point has a cluster label. Use for analysis or visualisation.

Key idea: Assign → Update centroid → Repeat until stable. This is the complete K-Means loop!

Euclidean Distance — How 'Nearest' is Measured

K-Means uses Euclidean Distance — the straight-line distance between two points. A point is assigned to whichever centroid has the SMALLEST distance to it.

$$d(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Worked example

Data point P = (2, 3)

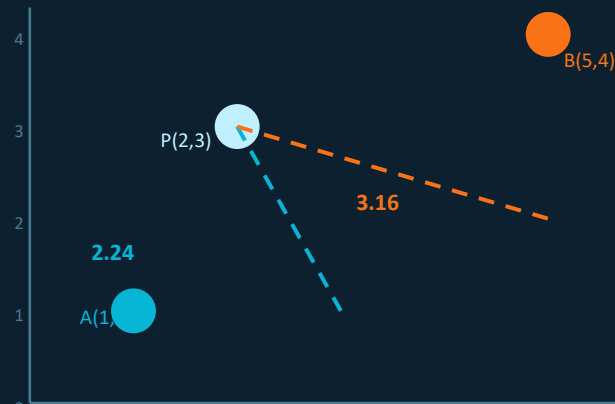
Centroid A = (1, 1) | Centroid B = (5, 4)

$$d(P, A) = \sqrt{[(2-1)^2 + (3-1)^2]} = \sqrt{[1+4]} = \sqrt{5} = 2.24$$

$$d(P, B) = \sqrt{[(2-5)^2 + (3-4)^2]} = \sqrt{[9+1]} = \sqrt{10} = 3.16$$

$d(P,A)=2.24 < d(P,B)=3.16 \rightarrow$ Point P assigned to Centroid A

Distance visualisation

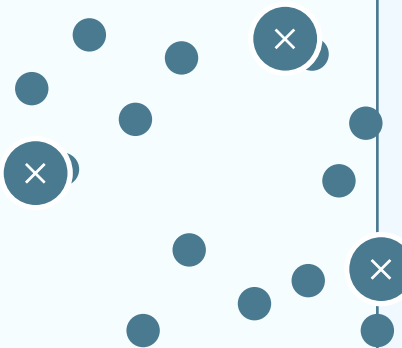


K-Means in Action — Watch It Learn (K=3)

Watch K-Means at each iteration. The algorithm keeps adjusting centroids until no point changes its cluster. This is called **CONVERGENCE**.

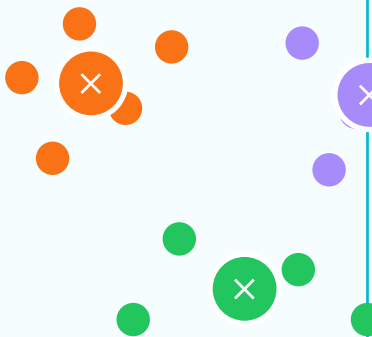
Step 0: Random centroids

3 centroids placed randomly (X)



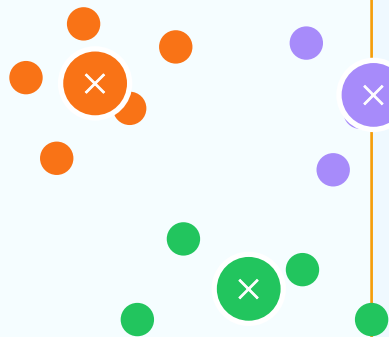
Step 1: Assign points

Each point → nearest centroid



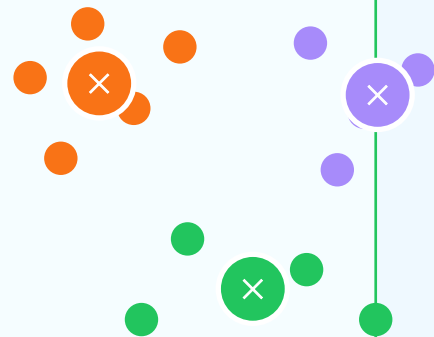
Step 2: Move centroids

Centroids move to cluster means



Step 3: Converged!

No changes — algorithm stops



Convergence = when NO point changes its cluster between two iterations — algorithm stops

Full Worked Example — Customer Segmentation

Dataset: 6 customers with Monthly Spend (₹000s) and Visit Frequency. Goal: Group into K=2 clusters.

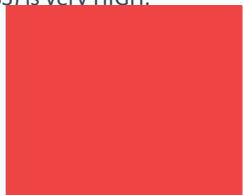
Customer	Spend (₹k)	Visits/mo	Initial Group	Final Cluster
C1	2	3	Cluster A	Cluster A (Low)
C2	3	2	Cluster A	Cluster A (Low)
C3	2	4	Cluster A	Cluster A (Low)
C4	8	9	Cluster B	Cluster B (High)
C5	9	8	Cluster B	Cluster B (High)
C6	7	10	Cluster B	Cluster B (High)

Convergence — When Does the Algorithm Stop?

Convergence happens when the algorithm has found stable clusters — no data point moves from one cluster to another between two consecutive iterations. The centroids stop moving.

Iteration 1

Centroids are random. Most points are assigned to wrong clusters. Loss (WCSS) is very HIGH.



WCSS

Iteration 2

Centroids move towards cluster centres. Some points reassigned. WCSS drops significantly.



WCSS

Iteration 3

Centroids almost stable. Very few points change cluster. WCSS still decreasing slowly.



WCSS

Iteration 5

No point changes cluster. Centroids are STABLE. WCSS has reached minimum. STOP!



WCSS

✓ CONVERGED

Three ways K-Means stops:

No reassignment: No point changes its cluster in an iteration — perfect convergence

Max iterations: Algorithm runs a maximum number of iterations (e.g. 300) and stops

Centroid movement $< \epsilon$: Centroids move less than a tiny threshold ϵ between iterations

How to Choose K — The Elbow Method

The biggest challenge in K-Means: you must set K before running! The Elbow Method helps. Run K-Means for K=1,2,3...10 and plot the WCSS (Within-Cluster Sum of Squares). Pick the K at the ELBOW.

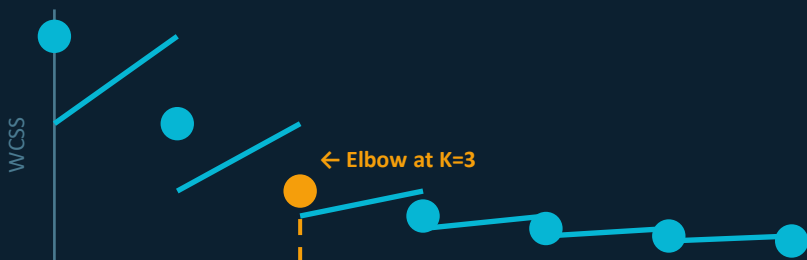
WCSS — Within-Cluster Sum of Squares

WCSS = total sum of squared distances from each point to its own centroid. Measures how tight the clusters are. LOWER WCSS = tighter, better clusters.

$$WCSS = \sum_k \sum_{i \in C_k} ||x_i - \mu_k||^2$$

where μ_k = centroid of cluster k

Elbow method plot (WCSS vs K)



How to use the Elbow Method

Run K-Means for K=1 to 10

Record WCSS for each K

Plot WCSS vs K graph

WCSS always decreases as K ↑

Find the 'elbow' point

Where curve bends sharply

Elbow Method: plot WCSS vs K → pick the K where the curve bends (elbow shape)

Evaluating K-Means & Its Limitations

Silhouette Score

$$s(i) = (b - a) / \max(a, b) \quad \text{Range: } -1 \text{ to } +1$$

a = avg distance to points in SAME cluster (cohesion)

b = avg distance to points in NEAREST other cluster (separation)

Score +1.0: Perfect — point fits well in its cluster

Score 0.0: Point is on the border between 2 clusters

Score -1.0: Point is in the WRONG cluster

WCSS (Inertia): sum of squared distances within each cluster. Lower = tighter clusters. Use to compare different K values (Elbow method).

Limitations of K-Means

✗ Must set K manually:

You must choose K before running. Wrong K = bad clusters.

✗ Sensitive to outliers:

One extreme point can badly pull a centroid away from the cluster.

✗ Assumes spherical clusters:

Fails for elongated, crescent, or ring-shaped clusters.

✗ Random initialisation:

Different starting centroids may give different final clusters.

✗ Feature scale sensitive:

Large-scale features dominate distance — always standardise first.

Fix: Use K-Means++ initialisation (smarter start) or try DBSCAN for non-spherical clusters.

Silhouette: closer to +1 = better | WCSS: lower = tighter | Always standardise features before K-Means

K-Means — Strengths, Weaknesses & Real Uses

Advantages

- ✓ Simple to understand: Easy to explain and implement — the core idea is just average distances
- ✓ Computationally fast: $O(n \times K \times i)$ — scales well to large datasets with millions of points
- ✓ Works well in practice: Despite simplicity, gives excellent results on many real problems
- ✓ Flexible K: You can try different values of K and compare cluster quality
- ✓ Easy to visualise: Clusters and centroids are easy to plot in 2D/3D for presentations

Disadvantages

- ✗ K must be predefined: If you pick wrong K, clusters are meaningless — use Elbow method
- ✗ Bad initialisation: Random start can lead to local minima — use K-Means++ instead
- ✗ Only spherical clusters: DBSCAN or Hierarchical handle arbitrary shapes better
- ✗ Outlier sensitive: Extreme values drag centroids — remove outliers before clustering
- ✗ Hard cluster boundaries: Every point must belong to exactly one cluster — no soft membership

C

Customer segmentation

Group shoppers: Premium, Regular, Budget

I

Image compression

Reduce 16M colours to K representative colours

D

Document clustering

Group news articles by topic automatically

A

Anomaly detection

Points far from all centroids = outliers

M

Medical imaging

Segment tissues in MRI/CT scans

Summary — K-Means Clustering

- 1 K-Means is an Unsupervised Learning algorithm — groups similar data without labels
- 2 K = number of clusters (you decide). Centroid = mean of all points in the cluster
- 3 Algorithm: Choose K → Place centroids → Assign points → Update centroids → Repeat → Stop
- 4 Distance formula: $d = \sqrt{[(x_2 - x_1)^2 + (y_2 - y_1)^2]}$ Each point goes to the NEAREST centroid
- 5 Convergence = no point changes cluster between iterations. Also stops at max iterations.
- 6 Elbow Method: plot WCSS vs K → pick the K at the elbow (where the curve bends)
- 7 Evaluate with: Silhouette Score (close to +1 = good) and WCSS (lower = tighter clusters)
- 8 Best for: Customer segmentation, Image compression, Document grouping, Anomaly detection

K-Means loop: Assign → Update Centroid → Repeat until stable | Always standardise features first!